

Apprentissage profond (Deep Learning I)

TP1 : Perceptron, ADALINE et Neurone Logistique

I. Introduction

Dans ce TP, nous allons explorer trois modèles fondamentaux de neurones artificiels : le **perceptron**, **ADALINE** et le **neurone logistique**. Ces modèles font partie des architectures les plus simples en apprentissage supervisé et constituent la base théorique des réseaux de neurones modernes.

Pour chacun d'entre eux, nous considérons un réseau de neurones très simple, composé de :

- Une couche d'entrée, correspondant aux caractéristiques des données (features)
- Une couche de sortie avec un seul neurone, qui produira la prédiction finale

Dans les trois cas, le neurone calcule d'abord une combinaison linéaire des entrées :

$$z = w \cdot x + w_0$$

Le biais w_0 joue un rôle similaire à l'ordonnée à l'origine dans une équation de droite : il permet au neurone de décaler la frontière de décision sans dépendre des entrées. Sans biais, la frontière passerait forcément par l'origine, ce qui limite la capacité du neurone à séparer correctement les données.

La principale différence entre ces modèles réside dans :

- la **fonction d'activation** utilisée ;
- la **fonction de coût** optimisée ;
- la **règle de mise à jour des poids**.

Plus précisément :

- Le **perceptron** utilise une fonction seuil (signe) et met à jour ses poids uniquement si la prédiction est incorrecte.
- **ADALINE** utilise une fonction identité et minimise l'erreur quadratique moyenne, ce qui permet une mise à jour continue des poids.
- Le **neurone logistique** utilise une fonction sigmoïde et minimise la cross-entropy, permettant d'obtenir une sortie interprétable comme une probabilité.

Les trois modèles étudiés n'utilisent pas la même représentation des classes.

- **Perceptron** : $y \in \{-1, +1\}$
- **ADALINE** : $y \in \{-1, +1\}$
- **Neurone logistique** : $y \in \{0, 1\}$

Cette différence est liée à la fonction de coût utilisée et à l'interprétation de la sortie du neurone. Les labels seront donc définis séparément dans chaque section correspondant au modèle étudié.

Pour rendre ces concepts plus intuitifs et permettre une visualisation directe, nous travaillerons sur des données bidimensionnelles. Chaque point représentera un exemple,

et sa position dans le plan correspondra aux deux caractéristiques utilisées comme entrée du neurone. Cela permettra de

- visualiser facilement les frontières de décision,
- de comparer le comportement des trois modèles
- analyser leur comportement sur des données linéairement séparables ou non.

II. Préparation des données d'apprentissage

Pour pouvoir visualiser facilement l'effet des neurones linéaires, nous allons travailler sur un dataset 2D. Chaque point représente un exemple et est défini par deux caractéristiques.

Nous allons créer deux groupes de points :

- **Groupe 0** : centré en (0,0)
- **Groupe 1** : centré en (2,2)

Chaque groupe sera affiché avec un symbole et une couleur différents afin de visualiser immédiatement leur position relative.

Les labels (y) ne sont pas encore instanciés numériquement ici. Ils seront créés séparément dans chaque section modèle conformément à la convention définie dans l'introduction.

Les trois modèles étudiés utiliseront exactement les mêmes ensembles d'apprentissage et de test afin de comparer leur performance.

Remplir les avec les bons appels et variables.

```
# Importer les librairies nécessaires numpy et matplotlib
import .....
# Fixer la reproductibilité
np.random.seed(42)

# Générer les points du premier groupe centrés autour de (0,0) (50 points)
G0 = np.random.randn(50, 2) + np.array([0,0])

# Générer les points du deuxième groupe centrés autour de (2,2) (50 points)
G1 = .....

# Combiner les points des deux groupes dans un seul tableau « dataset »
X = .....

# Visualiser les groupes dans un plan 2D
# Chaque groupe doit avoir un symbole et une couleur différents
plt.scatter(..., ..., color = ..., marker = ..., label = 'Groupe 0')
plt.scatter(..., ..., color = ..., marker = ..., label = 'Groupe 1')

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Dataset 2D linéairement séparable")
plt.legend()
plt.show()
```

- Modifier le nombre de points par groupe.
- Modifier les centres des groupes.
- Tester un cas où les centres sont plus proches afin d'obtenir un chevauchement important entre les classes (rendant la séparation linéaire difficile voire impossible)
- Observer visuellement les cas où une frontière linéaire semble difficile à tracer.
- Créez trois ensembles d'apprentissage distincts :
 1. **X** : linéairement séparable
 2. **X_ns** : non linéairement séparable
 3. **X_ls** : légèrement séparable (petite marge)
- Afficher les trois ensembles X, X_ns, X_ls dans une même figure.

III. Préparation des données de test

Dans nos expériences, les données de test ne sont pas un nouveau dataset réel. Nous construisons plutôt une **grille régulière de points** couvrant tout l'espace 2D.

Pourquoi ?

- Pour évaluer le modèle sur tout le plan
- Pour visualiser la frontière de décision
- Pour comprendre comment le neurone partitionne l'espace

On définit une grille régulière à l'aide de « **np.meshgrid** ».

```
# Affichage des point de test (les points de la grille)
def create_test_grid(X_train, resolution=0.2):
    """
    Crée une grille de points couvrant l'espace du dataset d'entraînement.
    X_train : données d'apprentissage (n_samples, 2)
    resolution : distance entre deux points de la grille

    Retourne :
    xx, yy : matrices meshgrid
    grid_points : tableau (n_points, 2)
    """
    # Définir les bornes
    x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
    y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1

    # Créer la grille
    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, resolution),
        np.arange(y_min, y_max, resolution)
    )

    # Transformer en liste de points
    grid_points = np.c_[xx.ravel(), yy.ravel()]

    return xx, yy, grid_points
```

La variable «**resolution**» contrôle la distance entre deux points consécutifs de la grille.

- Plus la résolution est grande, plus la grille est grossière (peu de points)
- Plus la résolution est petite, plus la grille est dense (beaucoup de points)

Voici la fonction pour l’affichage des points de test (points de la grille).

```
def plot_test_grid(grid_points):  
    """  
    Affiche uniquement les points de test (grille).  
    """  
    plt.figure()  
    plt.scatter(grid_points[:, 0], grid_points[:, 1], s=10)  
    plt.xlabel("x1")  
    plt.ylabel("x2")  
    plt.title("Points de test (grille)")  
    plt.gca().set_aspect('equal', adjustable='box')  
    plt.show()
```

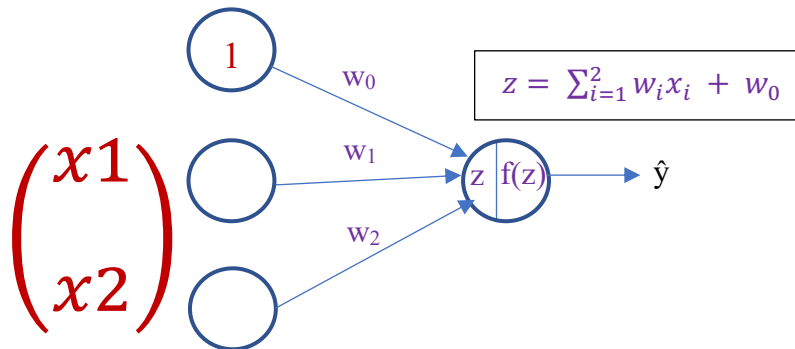
Une grille plus dense améliore la précision visuelle mais augmente le coût de calcul

- Tester plusieurs valeurs de résolution.
- Afficher la grille correspondante.
- Compter le nombre total de points de test (écrire une fonction **count_grid_points**).
- Comparer l’impact sur la densité de la grille.
- Créez trois ensembles de points de test en modifiant la résolution de la grille :
 - **Grille dense** : résolution très petite donc très grand nombre de points (le nombre de points croît quadratiquement lorsque la résolution diminue).
 - **Grille moyenne** : résolution intermédiaire pour réduire les temps de calcul
 - **Grille sparse** : résolution plus grande, ce qui donnera peu de points
- Affichez les trois grilles dans une même figure pour les comparer visuellement.
- Affichez une figure composée de 9 sous-graphes (3×3) correspondant à toutes les combinaisons possibles entre :
 - **Les trois ensembles d’apprentissage** :
 1. **X** : linéairement séparable
 2. **X_ls** : légèrement séparable
 3. **X_ns** : non linéairement séparable
 - **Les trois ensembles de test (grilles)** :
 1. Grille dense
 2. Grille moyenne
 3. Grille sparse

IV. Architecture commune et fonctions de base du neurone

Chaque modèle utilise un réseau très simple :

- **Couche d'entrée** : les coordonnées des points 2D
- **Une seule couche de sortie avec un seul neurone**



Propagation et activation

Pour chaque point x du dataset :

1. Combinaison linéaire :

$$z = w \cdot x + w_0$$

où w est le vecteur des poids et w_0 le biais.

La combinaison linéaire $z = w \cdot x + w_0$ signifie que le neurone calcule d'abord une somme pondérée des entrées. Par exemple, si $x = [2, 1]$, $w = [0.5, -1]$ et $w_0 = 0.1$, alors :

$$z = 0.5 \cdot 2 + (-1) \cdot 1 + 0.1 = 1.0 - 1 + 0.1 = 0.1$$

2. Sortie activée :

$$\hat{y} = f(z)$$

La fonction d'activation $f(z)$ dépend du modèle qu'on va construire :

- **Perceptron** : fonction **seuil (signe)** $\rightarrow \hat{y} \in \{-1, +1\}$
- **ADALINE** : fonction **identité** $\rightarrow \hat{y} \in \mathbb{R}$
- **Neurone logistique** : fonction **sigmoïde** $\rightarrow \hat{y} \in [0, 1]$

Variable	Signification	Domaine
z	Combinaison linéaire $w \cdot x + w_0$	$\mathbb{R}$
$\hat{y}$	Sortie du neurone $f(z)$	selon modèle
y_{pred}	Classe prédite	$\{-1, +1\}$ ou $\{0, 1\}$

3. Décision finale / prédiction :

On distingue :

- la sortie du neurone $\hat{y}$
- la décision de classification y_{pred}

• Perceptron

$$\hat{y} = \text{sign}(z)$$

La sortie est déjà discrète :

$$y_{\text{pred}} = \hat{y} \in \{-1, +1\}$$

• ADALINE

$$\hat{y} = z$$

La sortie est réelle.

La classification nécessite un seuillage explicite :

$$y_{\text{pred}} = \text{sign}(\hat{y})$$

$$y_{\text{pred}} \in \{-1, +1\}$$

• Neurone logistique

$$\hat{y} = \sigma(z)$$

La sortie est une probabilité $P(y=1|x)$.

La décision s'obtient par seuillage :

$$y_{\text{pred}} = \begin{cases} 1 & \text{si } \hat{y} \geq 0.5 \\ 0 & \text{sinon} \end{cases}$$

$$y_{\text{pred}} \in \{0, 1\}$$

Dans les formules mathématiques, nous noterons les labels y . Dans le code Python, nous utiliserons la variable y_true pour désigner les vraies étiquettes.

1. Initialisation des paramètres (les poids du réseau)

Avant l'apprentissage, il faut initialiser les poids w_i et le biais w_0 .

- Petites valeurs aléatoires, par exemple multipliées par 0.01
- Indépendant des labels
- Identique pour les trois modèles

À cette étape, on ne s'occupe ni des labels ni de la fonction d'activation, on prépare juste la structure du neurone.

```
# Fonction pour initialiser les poids et le biais
def initialize_weights(n_inputs) :
    #Créer un vecteur de poids w de taille n_inputs avec de petites valeurs aléatoires
    w = .....

    # Créer un biais b avec une petite valeur aléatoire
    b = .....
    return w, b
```

2. Propagation avant (forward pass)

Pour chaque point du dataset, le neurone calcule une combinaison linéaire des entrées :

$$z = w \cdot x + w_0 \quad \text{où } w \cdot x \text{ est le produit scalaire des deux vecteurs}$$

- z représente la valeur “avant activation” du neurone.
- Cette étape est identique pour les trois modèles.
- La sortie finale sera obtenue en appliquant ensuite la fonction d'activation spécifique à chaque modèle.

```
# Fonction pour calculer la sortie linéaire du neurone
def forward(x, w, b):

    # Calculer  $z = w \cdot x + b$  ; la variable b correspond au biais  $w_0$  ; w et x sont des
    # vecteurs de même dimension, b est un scalaire ; z = produit scalaire de w et x + b

    z = .....

    return z
```

3. Fonction d'activation

La fonction d'activation transforme la sortie linéaire du neurone ($z = w \cdot x + w_0$) en une sortie finale utilisable pour l'apprentissage ou la classification.

Dans cette partie commune du TP, nous préparons la structure de la fonction d'activation, mais les fonctions spécifiques à chaque modèle seront définies dans la section qui leur correspond :

- **Perceptron** : fonction signe (+1 ou -1)
- **ADALINE** : fonction identité ($\hat{y} = z$)
- **Neurone Logistique** : fonction sigmoïde ($1 / (1 + \exp(-z))$)

Pour l'instant, on crée une fonction générique où on prévoit la possibilité de passer la fonction d'activation comme paramètre pour rendre le code réutilisable et modulaire, ainsi la même structure peut être utilisée pour tous les modèles. Les fonctions spécifiques seront définies dans chaque section modèle.

```
# Fonction d'activation générique
def activation(z, activation_function) :
    # Appliquer la fonction d'activation fournie en paramètre
    return .....
```

4. Règle de mise à jour des poids

Dans les trois modèles étudiés (Perceptron, ADALINE, Régression Logistique), l'apprentissage consiste à **minimiser une fonction de coût** $\mathcal{L}(w, w_0)$.

On adopte une formulation générale basée sur la **descente de gradient** (ou sous-gradient) :

$$w \leftarrow w - \eta \nabla_w \mathcal{L}(w, w_0)$$

$$w_0 \leftarrow w_0 - \eta \frac{\partial \mathcal{L}}{\partial w_0}$$

où :

$\eta > 0$ est le taux d'apprentissage

$\nabla_w \mathcal{L}$ est le gradient par rapport aux poids

$\frac{\partial \mathcal{L}}{\partial w_0}$ est la dérivée par rapport au biais

a) Cas général

Pour une observation (x,y), on calcule :

$$z = w \cdot x + w_0$$

La fonction de perte dépend de y et de z : $\mathcal{L} = \mathcal{L}(y, z)$

En appliquant la règle de la chaîne :

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot x$$

$$\frac{\partial \mathcal{L}}{\partial w_0} = \frac{\partial \mathcal{L}}{\partial z}$$

La mise à jour des poids et du biais s'écrit alors de manière générale :

$$w \leftarrow w - \eta \cdot \left(\frac{\partial L}{\partial z} \cdot x \right)$$

$$w_0 \leftarrow w_0 - \eta \cdot \left(\frac{\partial L}{\partial z} \right)$$

Ainsi, toute la différence entre les modèles réside uniquement dans :

$$\frac{\partial L}{\partial z}$$

Pour simplifier le code et réutiliser la fonction `update_weights` on définit un schéma commun pour tous les modèles (le vecteur `x` n'apparaît plus explicitement ici car il est déjà inclus dans δ_w , qui dépend du modèle considéré) :

$$w \leftarrow w - \eta \delta_w$$

$$w_0 \leftarrow w_0 - \eta \delta_{w_0}$$

Chaque modèle définit δ_w et δ_{w_0} différemment :

Modèle	δ_w	δ_{w_0}
Perceptron	$-(y \cdot x)$ si mal classé	$-y$ si mal classé
ADALINE	$(\hat{y} - y) \cdot x$	$\hat{y} - y$
Neurone logistique	$(\hat{y} - y) \cdot x$	$\hat{y} - y$

Dans le cas batch (descente de gradient classique), les gradients sont moyennés sur l'ensemble des observations.

Cette séparation permet de réutiliser la fonction `update_weights` pour tous les modèles, tout en mettant en évidence les différences spécifiques à chaque modèle.

```
# Fonction commune pour mettre à jour les poids
def update_weights(w, b, delta_w, delta_b, learning_rate) :
    """
    w, b : poids et biais actuels
    delta_w, delta_b : corrections calculées pour le modèle spécifique
    learning_rate : taux d'apprentissage

    Retourne les poids et biais mis à jour
    """
    # delta_w et delta_b sont calculés dans les fonctions spécifiques à chaque
    # modèle
    # Ici on applique simplement la descente de gradient avec le learning_rate
    w = .....
    b = .....
    return w, b
```

5. Prédiction

Pour obtenir la sortie d'un neurone pour un ensemble de points, on combine la propagation et la fonction d'activation.

Pour chaque point x du dataset :

1. On calcule z avec la propagation
2. On applique la fonction d'activation
3. On stocke la prédiction

Cette fonction sera commune aux trois modèles, le comportement spécifique dépendra de la fonction d'activation passée en paramètre.

```
# Fonction pour prédire la sortie d'un neurone pour un ensemble de points
def predict(X, w, b, activation_function) :
    """
    X : tableau de points (exemples)
    w, b : poids et biais du neurone
    activation_function : fonction d'activation spécifique à chaque modèle
    """
    y_hats = []
    for x in X:
        z = ..... # Utiliser la fonction forward définie
        précédemment
        y_hat = .....
        y_hats.append(y_hat)
    return np.array(y_hats)
```

6. Calcul du coût (Loss)

Pour suivre l'apprentissage, il est utile de calculer une fonction de coût.

On distingue :

- **Erreur (error)** : quantité calculée pour une observation individuelle.
- **Coût (loss)** : agrégation des erreurs sur l'ensemble des observations du batch.

La définition du coût dépend du modèle :

- **Perceptron** : somme des erreurs de classification (nombre total d'exemples mal classés)
- **ADALINE** : moyenne des erreur quadratiques (MSE)
- **Neurone Logistique** : moyenne de la cross-entropy

Pour ADALINE et le neurone logistique, on utilise la moyenne sur l'ensemble le batch complet, afin que la valeur du coût ne dépende pas du nombre de données d'apprentissage.

Dans cette fonction commune, on calcule un coût générique qui sera défini précisément dans chaque section modèle.

Lors du calcul de la cross-entropy, des problèmes numériques peuvent apparaître. Mathématiquement, $\log(0)$ n'est pas défini et tend vers $-\infty$.

Si $\hat{y} = 0$ ou $\hat{y} = 1$ exactement, le coût devient infini.

Pour éviter cela, on introduit un très petit nombre $\text{eps}=1\text{e}-10$ et on contraint les valeurs de $\hat{y}$ dans l'intervalle $[\text{eps}, 1 - \text{eps}]$. Cela garantit la stabilité numérique du calcul du coût.

```
# Fonction générique pour calculer le coût
def compute_loss(y_true, y_hat, model_type) :
    """
    y_true : labels réels
    y_hat : sortie du neurone
    model_type : 'perceptron', 'adaline', 'logistic'

    Retourne la valeur du coût (loss) sur le batch
    """
    loss = None
    if model_type == 'perceptron' :
        # Nombre total d'exemples mal classés (somme)
        # loss = .....
        # À compléter après l'étude du Perceptron
        pass

    elif model_type == 'adaline' :
        # Moyenne des erreurs quadratique
        # loss = .....
        # À compléter après l'étude de l'ADALINE
        pass

    elif model_type == 'logistic' :
        # Moyenne de la cross-entropy
        # Si y_hat est nul on aura log(0), pour éviter cela on ajoute epsilon
        eps = 1e-10
        y_hat = np.clip(y_hat, eps, 1 - eps)
        # loss = .....
        # À compléter après l'étude du Neurone logistique
        pass

    return loss
```

7. Visualisation de la frontière de décision

Pour les datasets 2D, il est très utile de visualiser la frontière de décision du neurone. Nous utilisons à cet effet une « grille de points couvrant tout l'espace 2D », que l'on considère comme un ensemble de test artificiel. Chaque point de cette grille est évalué par le neurone afin de déterminer sa classe prédite. Lorsque le paramètre « resolution » est très petit, la grille devient plus dense, ce qui permet de visualiser « avec précision la frontière de décision » sur tout le plan. Cette fonction trace la ligne séparant les deux classes et affiche également les points d'entraînement pour comparaison.

```
def plot_decision_boundary(X, y_true, w, b, activation_function, model_type,
resolution=0.02):
    """
    Trace la frontière de décision d'un neurone simple sur un dataset 2D.

    X : tableau de points d'entraînement (n_samples, 2)
    y_true : labels réels (0/1 ou -1/+1 selon le modèle)
    w, b : poids et biais du neurone
    activation_function : fonction d'activation spécifique au modèle
    resolution : pas de la grille pour la visualisation

    Cette fonction réutilise les fonctions create_test_grid et predict pour générer
    la grille et prédire sur tous les points.

    """
    # Créer la grille de points sur tout l'espace
    xx, yy, grid_points = .....

    # Prédire la sortie pour chaque point de la grille
    Z = .....

    # Pour classification binaire, convertir en 0/1 si nécessaire
    if model_type == 'logistic':
        Z = np.where(Z >= 0.5, 1, 0)
    else:
        Z = np.where(Z >= 0, 1, -1)

    # Reshaper les prédictions pour correspondre à la grille
    Z = Z.reshape(xx.shape)
    # Tracer la frontière de décision
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    # Tracer les points réels
    plt.scatter(X[:, 0], X[:, 1], c=y_true, cmap=plt.cm.coolwarm, edgecolors='k',
s=50)

    plt.xlabel('x1')
    plt.ylabel('x2')
    plt.title('Frontière de décision du neurone')
    plt.show()
```

V. Le perceptron

Le perceptron est un neurone simple qui utilise :

- **Fonction d'activation** : fonction seuil (signe) $\rightarrow$ sortie +1 ou -1
- **Règle d'apprentissage** : mise à jour des poids uniquement si l'exemple est mal classé

On peut interpréter la règle du perceptron comme la minimisation d'une fonction de type hinge simplifiée $\max(0, -yz)$.

La fonction de coût du perceptron à minimiser pour l'obtention des poids optimaux est : $L = \max(0, -yz)$

Cette fonction est convexe mais non différentiable en $z = 0$.

On calcule le sous gradient :

$$\frac{\partial L}{\partial z} = \begin{cases} 0 & \text{si bien classé} \\ -y & \text{si mal classé} \end{cases}$$

Alors

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} x = -yx \quad (\text{uniquement si mal classé})$$

Donc,

$$\begin{aligned} \delta w &= -y \cdot x && \text{si mal classé, sinon 0} \\ \delta w_0 &= -y && \text{si mal classé, sinon 0} \end{aligned}$$

```
# Fonction d'activation spécifique au perceptron
def perceptron_activation(z):
    if z >= 0:
        return .....
    else:
        return .....

# Calcul de delta_w et delta_b pour le perceptron

def perceptron_delta(X, y_true, y_hat):
    """
    X : le dataset d'apprentissage
    y_true : label réel (+1/-1)
    y_hat : prédiction actuelle du neurone
    Retourne
        delta_w : moyenne des  $\delta w$  sur tous les points mal classé du batch
        delta_b : moyenne des  $\delta b$  sur tous les points mal classé du batch
        delta_w & delta_b seront utilisés par la fonction update_weights
    """
    n_samples = X.shape[0]
    delta_w_total = np.zeros(X.shape[1])
```

```

delta_b_total = 0
count_misclassified = 0

for i in range(n_samples) :
    #On utilise ici le nombre d'erreurs de classification, plus simple à interpréter.
    if y_hat[i] != y_true[i] :
        delta_w_total += .....
        delta_b_total += .....
        count_misclassified += 1

    # éviter la division par zéro si aucun point mal classé
    if count_misclassified > 0:
        delta_w = .....
        delta_b = .....
    else:
        delta_w = np.zeros_like(delta_w_total)
        delta_b = 0
    return ....., .....

```

Complétez maintenant la partie **perceptron** de la fonction **compute_loss**.

1. Boucle d'apprentissage du Perceptron

La boucle d'apprentissage du perceptron consiste à itérer sur l'ensemble du dataset pour ajuster les poids « w » et le biais « w_0 » afin de réduire les erreurs de classification.

À chaque epoch :

1. On calcule la sortie du perceptron pour tous les exemples.
2. On identifie les points mal classés et on calcule les mises à jour δ_w δ_{w_0} correspondantes.
3. On met à jour les poids et le biais en appliquant le « learning rate ».
4. On évalue le coût pour suivre la convergence.

Cette procédure se répète jusqu'à ce que les poids convergent ou qu'un nombre maximal d'epochs soit atteint, permettant au perceptron de séparer les classes linéairement séparables.

Le perceptron cherche une solution qui annule totalement l'erreur de classification. Si une telle solution n'existe pas (cas non linéairement séparable), l'algorithme continue indéfiniment à ajuster les poids sans convergence réelle.

```

# Création des labels (-1 pour G0, +1 pour G1)
y_true = np.array([-1]*len(G0) + [1]*len(G1))

# Initialisation
w, b = .....

learning_rate = 0.01
n_epochs = 50

```

```

# Boucle d'apprentissage
for epoch in range(n_epochs):
    # prédiction sur tout le dataset
    y_hat = predict(X, w, b, perceptron_activation)

    # calcul des deltas moyens
    delta_w, delta_b = perceptron_delta(X, y_true, y_hat)

    # mise à jour des poids et du biais
    w, b = update_weights(w, b, delta_w, delta_b, learning_rate)

    # optionnel : calcul du coût pour suivi
    loss = compute_loss(y_true, y_hat, model_type='perceptron')
    print(f'Epoch {epoch+1} - coût : {loss}')

```

2. Test du modèle sur les données de test.

Après l'apprentissage du modèle, il est essentiel d'évaluer son comportement sur de la base de test (nouveaux points de l'espace d'entrée non vus lors de l'apprentissage).

Lorsque la résolution de la grille est suffisamment fine, l'ensemble des points colorés permet de visualiser pratiquement tout l'espace 2D et met en évidence la frontière de décision apprise par le modèle.

```

# Générer la grille
xx, yy, grid_points = create_test_grid(X, resolution=0.5)

# Prédire la classe de chaque point
grid_predictions = .....

# Séparer selon la classe
grid_class_neg = grid_points[grid_predictions == ..... ]
grid_class_pos = grid_points[grid_predictions == ..... ]

# Affichage
plt.figure(figsize=(8,6))

#Affichage, en bleu clair, des points de test dont la classe prédite est -1
plt.scatter(.....)

#Affichage, en rose, des points de test dont la classe prédite est +1
plt.scatter(.....)

# Affichage, en bleu, des données d'apprentissage labélisées en -1
plt.scatter(.....)

# Affichage, en rouge, des données d'apprentissage labélisées en +1
plt.scatter(.....)

```

```
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Prédiction des points de test")
plt.legend()
plt.show()
```

3. Analyse de la frontière de décision du Perceptron

Après plusieurs itérations d'apprentissage, nous pouvons visualiser la frontière de décision obtenue par le perceptron.

Cette frontière correspond à l'ensemble des points vérifiant :

$$w \cdot x + w_0 = 0$$

En dimension 2, cela donne :

$$w_1 x_1 + w_2 x_2 + w_0 = 0$$

On peut réécrire cette équation sous la forme

$$x_2 = -\frac{w_1}{w_2} x_1 - \frac{w_0}{w_2}$$

Elle définit une droite dans $\mathbb{R}^2$ qui sépare les deux classes. Les poids w déterminent l'orientation de la droite et le biais w_0 détermine son déplacement. Si $w_2 \approx 0$, la frontière est quasi verticale. Pour l'affichage de la frontière faites appel à la fonction « `plot_decision_boundary` ».

```
plot_decision_boundary(.....)
```

À partir des poids finaux obtenus :

- Affichez les valeurs de w_1 , w_2 et w_0 .
- Vérifiez que la droite tracée correspond bien à l'équation ci-dessus.
- Modifiez légèrement les poids à la main : que constatez-vous ?

4. Affichage dynamique de la frontière de décision

Pendant l'apprentissage du perceptron, les poids w et le biais w_0 changent à chaque epoch. Chaque modification fait bouger la frontière de décision. En dimension 2, cette frontière correspond à la droite décrite en détail dans la section précédente « *Analyse de la frontière de décision du Perceptron* ».

Dans cette section, nous allons créer une [animation](#) montrant comment la droite évolue, pour mieux visualiser le processus d'apprentissage.

a) Sauvegarder les paramètres du modèle

Afin de pouvoir reconstruire la frontière pour l'animation, il faut garder une trace des poids et du biais à chaque epoch.

- Initialiser deux listes avant la boucle d'apprentissage pour sauvegarder les paramètres du modèle (w et w_0)

```
# Une liste pour sauvegarder les poids w ( $w_1$  et  $w_2$ )
w_history = []
# Une liste pour sauvegarder les biais b ( $w_0$ )
b_history = []
```

- Sauvegarder les poids et biais dans les listes initialisées après chaque mise à jour dans la boucle d'apprentissage.

```
# Boucle d'apprentissage
for epoch in range(n_epochs):
    -----
    -----
    -----
    # mise à jour des poids et du biais
    w, b = update_weights(w, b, delta_w, delta_b, learning_rate)
    # Sauvegarde des paramètres pour l'animation
    w_history.append(w.copy())
    b_history.append(b)
    -----
    -----
```

b) Affichage des points d'apprentissage

Avant de tracer la droite qui sépare les deux classes, on visualise les points de l'ensemble d'apprentissage en représentant les points de chaque classe par une couleur et un symbole distinct.

```
# Création de la figure
fig, ax = plt.subplots(figsize=(8,6))
ax.set_xlim(X[:,0].min()-1, X[:,0].max()+1)
ax.set_ylim(X[:,1].min()-1, X[:,1].max()+1)
# Affichage des points d'apprentissage
ax.scatter(X[y_true==-1,0], X[y_true==-1,1], color='blue', label='Classe -1',
           edgecolor='k', marker='o')
ax.scatter(X[y_true==1,0], X[y_true==1,1], color='red', label='Classe +1',
           edgecolor='k', marker='D')
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.legend()
#plt.show() Si vous souhaitez l'afficher avant l'ajout de la droite
```

c) Calcul de la droite pour chaque epoch

Chaque epoch correspond à une nouvelle droite.

$$w_1x_1 + w_2x_2 + w_0 = 0 \Leftrightarrow x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

On utilisera les poids sauvegardés pour calculer les paramètres de la droite à

tracer ($y = mx - p$ avec $m = -\frac{w_1}{w_2}$ et $p = -\frac{w_0}{w_2}$) à chaque epoch.

```
x_vals = np.array([X[:,0].min(), X[:,0].max()])
y_vals = - (w_history[epoch][0]/w_history[epoch][1]) * x_vals -
           (b_history[epoch]/w_history[epoch][1])
```

d) Animation matplotlib

Afin de créer une figure animée, on utilisera la méthode FuncAnimation qui a besoin des fonction init et update.

```
# Crée l'objet graphique qui représente la ligne dans la figure, au départ la
# ligne est vide
line, = ax.plot([], [], 'k--', lw=2, label='Frontière apprentissage')

# Initialise le contenu de cet objet
def init():
    line.set_data([], [])
    return line,

# Met à jour le contenu de cet objet à chaque frame
def update(epoch):
    x_vals = np.array([X[:,0].min(), X[:,0].max()])
    y_vals = - (w_history[epoch][0]/w_history[epoch][1]) * x_vals -
              (b_history[epoch]/w_history[epoch][1])
    line.set_data(x_vals, y_vals)
    return line,

# Création de la l'animation image par image (frame par frame)
ani = animation.FuncAnimation(fig, update, frames=len(w_history),
                              init_func=init, blit=True, interval=500)

#Affichage de l'animation
plt.show()
```

e) Export HTML

Afin de visualiser l'animation directement dans Jupyter Notebook.

```
# Affichage dans le notebook
from IPython.display import HTML
HTML(ani.to_jshtml())
```

f) Export GIF

Afin de créer un fichier GIF montrant l'évolution de la frontière.

```
# Sauvegarde dans une image gif  
ani.save('perceptron_evolution.gif', writer='pillow')
```

5. Limites du Perceptron

Après avoir visualisé la frontière de décision obtenue sur les données d'apprentissage et sur la grille de test, nous pouvons analyser le comportement du perceptron. Le perceptron minimise une fonction de coût convexe mais non différentiable. La convergence n'est garantie que si les données sont linéairement séparables. Il cherche uniquement à annuler les erreurs de classification.

a) Questions d'analyse :

- La frontière de décision est-elle bien une droite ?
- Cette droite semble-t-elle correctement séparer les deux classes ?
- Comparez la frontière obtenue sur :
 - les données linéairement séparables
 - les données non séparables
- Le coût devient-il nul dans le cas linéairement séparable ?
- Que se passe-t-il lorsque les données ne sont pas séparables ?
 - Le coût oscille-t-il ?
 - Les poids semblent-ils stabiliser ?
- Modifiez le nombre d'époques (par exemple 5, 10, 20, 50, 100) et observez l'évolution du coût et la stabilisation ou l'oscillation des poids.
- Le nombre d'itérations influence-t-il le résultat final ?
- Modifiez le taux d'apprentissage η (par exemple 0.001, 0.01, 0.1) et notez l'impact sur la vitesse de convergence et la stabilité de la frontière de décision.
- Le taux d'apprentissage influence-t-il la stabilité ?
- Tracez l'évolution du coût au fil des époques pour chaque combinaison, et comparez les résultats.

VI. L'ADALINE

ADALINE (ADaptive LINEar NEuron) utilise :

- **Fonction d'activation** : identité ($\hat{y} = z$)
- **Règle d'apprentissage** : descente de gradient sur l'erreur quadratique moyenne « MSE »

ADALINE est conçu pour des problèmes de régression linéaire et produit une sortie réelle ; la classification nécessite un seuillage explicite. ADALINE cherche à minimiser une fonction de coût différentiable. La fonction de coût à minimiser pour l'obtention des poids optimaux est (le facteur 1/2 simplifie la dérivation) :

$$L = \frac{1}{2} (y - \hat{y})^2$$

On calcule le gradient de la fonction de coût :

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x$$

$$\frac{\partial L}{\partial w_0} = (\hat{y} - y)$$

Contrairement au perceptron :

- ADALINE calcule l'erreur sur l'ensemble des données d'apprentissage,
- la mise à jour des poids dépend de l'erreur numérique $\hat{y} - y$,
- les poids sont mis à jour même si le point est correctement classé.

En effet, même si un point est du bon côté de la frontière, la valeur prédite $\hat{y}$ peut être éloignée de la valeur cible y . L'erreur quadratique étant continue, elle peut être non nulle même en cas de classification correcte. La fonction MSE est convexe pour un modèle linéaire, ce qui garantit l'existence d'un minimum global.

Ainsi les deltas sont calculés pour **tous les exemples**, et non uniquement pour les points mal classés :

$$\delta_w = (\hat{y} - y) \cdot x$$

$$\delta_{w_0} = (\hat{y} - y)$$

Contrairement au perceptron, la mise à jour des poids est continue et dépend de l'erreur numérique.

```
# Fonction d'activation spécifique à ADALINE
def adaline_activation(z):
    return .....

# Calcul de delta_w et delta_b pour ADALINE
def adaline_delta(X, y_true, y_hat):
    """
    X : le dataset d'apprentissage
    Y_true: label réel (0/1 ou -1/+1)
    y_hat : sortie linéaire du neurone
    Calcule les deltas moyens sur le batch complet.
    Retourne delta_w et delta_b
    """
    errors = y_hat - y_true
    delta_w = .....
    delta_b = .....
    return ....., .....
```

Complétez maintenant la partie **ADALINE** de la fonction **compute_loss**.

1. Boucle d'apprentissage de l'ADALINE

```
# Création des labels (-1 pour G0, +1 pour G1)
y_true = np.array([-1]*len(G0) + [1]*len(G1))

# Initialisation
w, b = .....

learning_rate = 0.01
n_epochs = 50

# Boucle d'apprentissage
for epoch in range(n_epochs):
    # prédiction sur tout le dataset
    y_hat = .....

    # calcul des deltas moyens
    delta_w, delta_b = .....

    # mise à jour des poids et du biais
    w, b = .....

    # optionnel : calcul du coût pour suivi
    loss = .....

    print(f'Epoch {epoch+1} - coût : {loss}')
```

En vous appuyant sur la boucle d'apprentissage du perceptron :

1. Créez les labels pour ADALINE (-1 pour G0, +1 pour G1).
2. Initialisez les poids.
3. Implémentez la boucle d'apprentissage :
 - calcul de $\hat{y}$ avec `adaline_activation`
 - calcul des deltas avec `adaline_delta`
 - mise à jour des poids avec `update_weights`
 - calcul du coût avec `compute_loss(..., model_type='adaline')`
4. Tracez l'évolution du coût au fil des epochs.
5. Affichez la frontière de décision avec `plot_decision_boundary`.
6. Le coût devient-il nul ?
7. Comparez la convergence avec celle du perceptron.
8. Les poids semblent-ils plus stables ?
9. Que se passe-t-il sur un dataset non linéairement séparable ?
10. La frontière obtenue est-elle différente de celle du perceptron ?

VII. Le neurone logistique (régression logistique)

Le neurone logistique est un modèle de classification binaire probabiliste :

- **Fonction d'activation** : sigmoïde

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

La sortie est interprétable comme une probabilité $P(y = 1 | x)$.

- **Règle d'apprentissage** : descente de gradient sur la cross-entropy, qui est adaptée à l'estimation de probabilités et à l'apprentissage par maximum de vraisemblance sous hypothèse de modèle Bernoulli.

Même si la sortie est non linéaire (sigmoïde), la frontière de décision reste linéaire, définie par :

$$w \cdot x + w_0 = 0$$

La probabilité $\hat{y}$ évolue de manière non linéaire autour de cette frontière.

La fonction de coût à minimiser pour l'obtention des poids optimaux est :

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

En combinant sigmoïde + cross-entropy, le gradient se simplifie et sa forme devient identique à celui d'ADALINE. La mise à jour des poids s'écrit :

$$\begin{aligned}\delta w &= (\hat{y} - y) \cdot x \\ \delta w_0 &= (\hat{y} - y)\end{aligned}$$

Contrairement au perceptron et à ADALINE :

- la sortie est interprétée comme une **probabilité** $P(y = 1 | x)$,
- les labels doivent être définis dans $\{0,1\}$,
- la décision finale s'obtient par seuillage à 0.5.

```
# Fonction d'activation sigmoïde
def logistic_activation(z):
    return .....

# Calcul de delta_w et delta_b pour le neurone logistique
def logistic_delta(X, y_true, y_hat):
    """
    X : le dataset d'apprentissage
    y_true : label réel (0/1)
    y_hat : sortie du neurone (probabilité)
    Retourne delta_w et delta_b
    """
    errors = y_hat - y_true
    delta_w = .....
    delta_b = .....
    return ....., .....
```

Complétez maintenant la partie **logistique** de la fonction **compute_loss**.

En vous appuyant sur la structure utilisée pour ADALINE :

1. Créez les labels pour la régression logistique (0 pour G0, 1 pour G1).
2. Initialisez les poids.
3. Implémentez la boucle d'apprentissage :
 - calcul de $\hat{y}$ avec **logistic_activation**
 - calcul des deltas avec **logistic_delta**
 - mise à jour avec **update_weights**
 - calcul du coût avec **compute_loss(..., model_type='logistic')**
4. Tracez l'évolution de la **cross-entropy** au fil des epochs.
5. Affichez la frontière de décision avec **plot_decision_boundary**.
6. La sortie $\hat{y}$ est-elle toujours comprise entre 0 et 1 ?
7. Comparez la courbe du coût avec celle d'ADALINE.
8. Le modèle converge-t-il sur un dataset non linéairement séparable ?
9. La frontière de décision est-elle différente de celle d'ADALINE ?
10. Pourquoi la cross-entropy est-elle plus adaptée que la MSE pour un problème probabiliste ?